PORTFOLIO

이동호 | Python Backend Developer

9년 동안 Python으로 웹 백엔드, 비동기 작업 시스템과 데이터 수집 서비스를 개발했습니다. Django·Flask API부터 Celery 작업 처리, AWS 인프라, 세무 신고·ERP 연동과 내부 운영용 AI 에이전트까지 담당했습니다.

혜움랩스 · Python 개발자 · 정규직 | 2017.04 - 2026.07

지원 분야: Python Backend / 주요 분야: 웹 백엔드 · 비동기 처리 · 데이터 수집

경력 요약

Django·Flask 기반 웹 백엔드, 데이터 모델과 API 개발

Celery·Redis 기반 비동기 작업 시스템 개발 및 장기 운영

PostgreSQL·DynamoDB·RDS를 사용한 수집 데이터 저장 구조 설계·변경

AWS Lambda 서버리스 애플리케이션과 외부 서비스 연동 기능 개발

ECS·Docker·Terraform을 사용한 Linux 기반 스크래핑 서비스 리뉴얼

CloudWatch·Slack을 사용한 실행 상태, 로그, 오류 모니터링 구성

CS·운영 데이터를 활용한 내부 지원 AI 에이전트 개발

기술

Backend	Python, Django, Flask, ORM, REST API
Data	PostgreSQL, RDS, DynamoDB, S3, Data Modeling
Async / Messaging	Celery, Redis, SQS
AWS	ECS, EC2, Lambda, CloudWatch
Infra / CI	Docker, Terraform, GitHub Actions, Linux, Windows Server
Integration	Web Scraping, C++ DLL API, Google API, Slack, KakaoTalk, AI Agent

학력

세종대학교 경영학과 · 졸업 | 2006.03 - 2013.06

연락처 및 웹 포트폴리오

loqbzes@gmail.com
+82 10-3272-8012
https://dongho-lee-portfolio.loqbzes.chatgpt.site

PROJECT 01 2026.03 - 2026.06

내부 CS·운영 지원 AI 에이전트 개발

주요 결과

개발자에게 바로 전달되던 CS 문의가 AI 응답을 먼저 거치는 흐름으로 전환

기존 문제

복잡한 세무 도메인과 내부 시스템으로 인한 CS·운영 담당자의 학습 기간 장기화

문의마다 개발자가 관련 코드와 과거 응답 기록을 직접 찾아야 하는 구조

실무 판단이 필요한 문의에 개발자가 답변하기 어려운 상황 발생

담당 범위

애플리케이션 개발 전 과정을 단독으로 담당했습니다. 배포에는 기존 프로젝트에 구성되어 있던 인프라와 CI/CD를 활용했습니다.

처리 구조

01 개발진에 전달할 CS를 담당자가 선택하거나 스레드에서 AI를 멘션하면 Slack Bolt가 이벤트와 전체 스레드 내용을 수신

02 Claude Agent SDK가 에이전트를 실행하고, 질문에 따라 필요한 정보원을 도구로 자율 선택

03 일별·월별로 수집해 S3에 Markdown으로 저장한 과거 질의응답, 주요 소스 코드, 내부 시스템 조회 API를 사용

04 조회 결과를 같은 Slack 스레드에 바로 응답

05 Slack 멘션으로 코드 수정을 명시적으로 요청하고 안전장치 조건을 충족한 경우에만 소스 코드를 수정해 별도 브랜치와 Draft PR을 생성

코드 수정 조건

기능 변경으로 판단되는 요청은 코드 수정을 거절

수정 범위가 100줄을 초과하면 작업을 거절

자동 반영하지 않고 Draft PR 생성까지만 수행하며 최종 리뷰는 개발자가 담당

도입 결과

CS·운영 담당자의 긍정적인 사용 반응

다른 사업부의 Slack 기반 AI 에이전트 개발로 확산

기술 스택

Python | Slack Bolt | Claude Agent SDK | S3 | ECS | GitHub

PROJECT 02 2020.06 - 2026.07

세무 정보 스크래핑 서비스 개발·리뉴얼

주요 결과

Linux 전환과 동시 처리량 확대를 함께 적용해 부가세 자료 스크래핑 시간을 약 4시간에서 1시간 30분으로 단축

프로젝트 범위

2020년부터 Windows 기반 서비스의 개발·운영을 담당했으며, 2024년부터 Linux·ECS 기반 리뉴얼을 진행했습니다.

리뉴얼 전 개발·개선

- 세무 일정에 따라 필요한 스크래핑·신고 서비스를 개발하고 운영
- 여러 Lambda 프로젝트에 분산돼 있던 스크래핑 후처리 기능과 실행 흐름을 파악해 유지보수
- 추상 태스크 기반 구조를 도입해 공통 실행 흐름과 세목별 고유 로직을 분리하고 신고·스크래핑 정보를 모델링

기존 문제

Windows 기반 실행 환경으로 인한 오토 스케일링 제약

긴 개별 스크래핑 처리 시간과 제한된 동시 처리량

작업을 여러 구간으로 나누는 우회 방식을 적용해도 추가 처리가 불가능한 용량 한계

담당 범위

리뉴얼 프로젝트 생성부터 Linux 스크래핑 코드, Celery·Redis 작업 처리, Docker·ECS·오토 스케일링, Terraform 인프라와 CI/CD 구축까지 전 과정을 직접 담당했습니다.

세무 정보 스크래핑 서비스 개발·리뉴얼 (계속)

처리 구조

01 스크래핑 코드를 Linux 환경에서 실행하도록 변경해 개별 작업의 처리 시간을 단축

02 스크래핑 요청을 Celery 태스크로 발행하고 ECS Task에서 실행되는 Celery 워커가 처리

03 ElastiCache for Redis를 Celery 브로커로 사용하고 Celery Result Backend에 작업 결과와 상태를 저장

04 ECS Task의 CPU 사용량을 기준으로 실행 Task 수를 자동 조정해 동시 처리량을 확대

05 staging 브랜치 대상 PR에서 테스트를 실행하고, GitHub Actions에서 Docker 이미지 빌드·ECR 업로드·ECS 배포를 수행

운영 및 모니터링

Celery Flower로 워커와 태스크 상태 확인

CloudWatch Logs로 실행 로그 수집

Sentry로 예외 수집과 오류 상황 추적

실패한 작업은 운영자가 수동 실행하거나 사용자가 직접 재시도

도입 결과

세무 일정에 맞춰 약 1년간 기능을 단계적으로 이전

Windows가 반드시 필요한 일부 작업을 제외한 모든 스크래핑을 신규 환경으로 전환

기술 스택

Python · Celery · ElastiCache for Redis · ECS · ECR · Docker · Terraform · GitHub Actions · CloudWatch · Sentry

PROJECT 03 2023.01 - 2026.07

세무사랑 ERP API 연동

주요 결과

SmartA 업데이트 종료에 대응해 기존 ERP 업무 전체를 세무사랑 환경으로 이전

기존 문제

SmartA에서 수행하던 ERP 업무를 세무사랑 환경으로 이전할 필요

제공된 DLL을 로드할 수 있는 최신 Python 환경이 3.6 32-bit여서 기존 애플리케이션에서 직접 호출할 수 없는 제약

데이터 조회·저장은 API로 가능했지만 일부 API는 실제 ERP 화면 실행이 필수

담당 범위

PM과 함께 필요한 API와 파라미터를 산출해 세무사랑 측에 전달할 개발 요청 문서를 작성했으며, DLL 호출 애플리케이션부터 기존 시스템 연동과 UI 자동화까지 모든 개발을 직접 담당했습니다.

처리 구조

01 ERP 데이터 조회·저장 작업은 세무사랑 API로 처리하고 화면이 필요한 기능만 UI 자동화로 보완

02 C++ DLL을 로드할 수 있는 Python 3.6 32-bit 전용 CPython 애플리케이션을 별도로 구성

03 기존 애플리케이션에서 어댑터를 통해 DLL 호출 애플리케이션의 Celery 태스크를 요청

04 DLL 호출 앱의 작업을 기존 앱의 큐와 분리해 Windows 로컬 Redis 큐와 Celery 워커에서 처리

05 로그인 태스크 실행 후 필요한 경우 UI 자동화를 수행하고, 이어서 API 호출 태스크를 실행

기술 스택

Python 3.6 32-bit · CPython · C++ DLL API · Celery · Redis · Windows UI Automation · EC2 Windows Server

PROJECT 04 2025.08

수임 정보 저장·조회 구조 개선 - DynamoDB에서 PostgreSQL로 전환

주요 결과

DynamoDB 테이블과 회사 정보 연동용 예약 Lambda를 제거해 비용과 처리 단계 축소

기존 문제

당시 수임 정보 서비스의 DynamoDB 테이블·인덱스 운영 비용 부담

새로운 조회 요구사항에 대응할 때 별도 인덱스 구성과 추가 개발이 필요했던 데이터 구조

DynamoDB 데이터와 회사 정보를 연결하기 위한 예약 Lambda 태스크 운영

담당 범위

기존 Django 프로젝트에서 PostgreSQL 데이터 모델, 스크래핑 결과 적재 로직과 API를 직접 구현했습니다. 기존 인프라와 배포 구성은 변경하지 않았습니다.

처리 구조

01 매일 수행되는 전체 스크래핑의 결과를 PostgreSQL에 직접 적재

02 사업자등록번호가 일치하는 스크래핑 데이터와 기존 회사 데이터를 Foreign Key로 연결

03 별도 데이터 마이그레이션 없이 신규 구조 적용 후 다음 전체 스크래핑부터 PostgreSQL 데이터로 전환

04 Django API 안에서 SQL 쿼리로 회사와 스크래핑 데이터를 연결해 홈택스·4대보험 수임 여부를 확인

도입 결과

수임 여부 확인 흐름을 Django 백엔드 내부의 관계형 조회로 단순화

기술 스택

Python Django ORM PostgreSQL DynamoDB AWS Lambda

PROJECT 05 2017.04 - 2023.07

더존 SmartA ERP 자동화 봇 개발

주요 결과

수임사의 손익 데이터를 제공하는 초기 서비스를 통해 고객 확보에 기여

프로젝트 목표

회사 설립 시점부터 각종 SmartA ERP 작업을 자동화하는 서비스 구축

초기 손익 데이터 제공 서비스에서 시작해 회사 성장에 맞춰 자동화 대상 업무를 지속적으로 확대

담당 범위

재무정보 추출, 급여 데이터 입력, 세무 증명서 PDF 출력과 SmartA 데이터 스크래핑을 포함해 프로젝트의 개발·개선·운영 전반을 단독으로 담당했습니다.

도입 결과

급여 입력 등 다양한 ERP 업무로 자동화 범위를 확대하며 회사의 서비스 확장에 기여

SmartA 업데이트 종료에 맞춰 대상 업무 전체를 세무사랑 기반으로 이전

기술 스택

Python Celery Redis EC2 Windows Server S3 CloudWatch Slack

PROJECT 06

개인 프로젝트 · 2026.09 - 개발 중

겨울의 안쪽 / Inside Winter

프로젝트 목표

덱빌딩과 이야기 선택을 결합한 1인용 미스터리 게임 개발

덱 구성·카드 사용·선택 이력에 따라 달라지는 이야기 분기와 다중 엔딩 구현

담당 범위

게임 기획·규칙·스토리 방향을 결정하고, 코드 구현과 이미지·음악 제작에는 AI를 활용했습니다. 직접 플레이하며 기능과 연출을 검수하고, 발견한 문제와 개선 요구사항을 구체화해 반복적으로 수정했습니다.

주요 구현

01 장면·선택지·엔딩을 JSON으로 관리하고 화면과 게임 규칙을 분리

02 카드별 고유 ID와 위치를 추적해 카드 효과·이동·제거 처리

03 진행·책갈피 저장과 복원, 플레이어별 기록 분리 및 중복 실행 방지

04 이야기 경로 그래프·도감·엔딩 갤러리와 자산 비동기 로딩·캐시 구성

05 카드 규칙·저장 복원·분기 연결에 대한 회귀 테스트와 실행판 검증 도구 구성

현재 상태

Windows 플레이 가능 버전을 제작했으며, 상용화를 위한 행정 절차를 진행하고 있습니다.

기술 스택

Godot · GDScript · Python · JSON · PowerShell · Codex